

目 录

1 第一部分 基于 face-api.js 的人脸分析实验

1.1 实验背景与目标

人脸分析是计算机视觉在人机交互、身份认证、内容理解和智能终端中的典型应用。传统人脸分析系统往往依赖后端服务完成模型推理，浏览器端只负责采集图像和展示结果。随着深度学习前端推理框架的发展，预训练模型可以直接在浏览器中加载并执行，降低了演示系统部署成本，也便于观察完整的人脸检测、属性估计和结果可视化流程。

本实验基于 `code/face-recognition/` 项目完成。该项目是一个纯前端人脸分析演示系统，核心依赖为 `face-api.js[?]`，界面层使用 Bootstrap 与 MDB，业务逻辑集中在 `dist/` 目录下的脚本中。实验目标包括：第一，实现图片上传后的人脸表情识别；第二，实现图片上传后的年龄与性别估计；第三，实现浏览器摄像头视频流中的实时人脸检测、关键点定位和表情追踪；第四，分析该系统的结构、运行流程和实验特点。

1.2 项目结构与实验环境

项目采用静态网页组织方式，不需要单独的后端服务。主要页面、脚本和资源的对应关系如表 ?? 所示。

表 1.1 人脸分析实验页面与脚本对应关系

功能页面	业务脚本	主要功能
<code>face_expression_recognition.html</code>	<code>dist/expression_script.js</code>	图片表情识别
<code>age_gender_estimation.html</code>	<code>dist/estimation_script.js</code>	年龄与性别估计
<code>webcam_face_tracking.html</code>	<code>dist/webcam_script.js</code>	摄像头实时追踪

模型文件位于 `models/` 目录,包括 `tiny_face_detector`、`face_landmark_68`、`face_expression`、`age_gender`、`face_recognition` 和 `ssd_mobilenetv1` 等预训练权重。示例数据位于 `public/` 目录,其中 `public/img/labeled_images/` 下包含 Chandler、Joey、Monica、Phoebe、Rachel、Ross 六类人物标注图片,可用于人脸识别演示。当前三个中文页面的脚本均使用 `MODEL_URL = "./models"` 从本地加载模型,因此实验运行不依赖远程模型地址,稳定性优于完全依赖外部 CDN 的演示方式。

1.3 核心方法设计

系统整体流程可以概括为“加载模型、获取输入、执行推理、绘制结果、同步文本展示”五个阶段。

首先,页面通过 `script` 标签引入 `dist/face-api.min.js` 和对应业务脚本。业务脚本启动时调用 `faceapi.nets.*.loadFromUri(MODEL_URL)` 并使用 `Promise.all` 并行加载所需网络。图片表情识别页面加载人脸检测、68 点关键点、人脸描述子、表情识别和 SSD MobileNet 模型;年龄性别估计页面加载检测、关键点、人脸描述子、年龄性别和 SSD

MobileNet 模型；摄像头追踪页面加载 Tiny Face Detector、关键点、人脸描述子和表情识别模型。

其次，图片类页面监听 `input[type=file]` 的 `change` 事件，将用户上传的文件转换为浏览器图像对象。脚本使用 `faceapi.createCanvasFromMedia(image)` 创建与图片尺寸一致的覆盖画布，并通过 `faceapi.matchDimensions` 保证检测框坐标和显示尺寸一致。摄像头页面则通过 `navigator.mediaDevices.getUserMedia` 或旧版 `getUserMedia` 兼容接口获取视频流，将视频流绑定到 `video.srcObject`。

最后，脚本调用 `detectAllFaces` 完成检测，并按功能继续串联 `withFaceLandmarks`、`withFaceExpressions` 或 `withAgeAndGender`。推理结果经过 `faceapi.resizeResults` 映射到显示尺寸后，一方面通过 `faceapi.draw` 绘制检测框、关键点和表情标签，另一方面渲染为侧边栏文本卡片，便于观察每张人脸的主表情、各表情概率、估计年龄、性别和置信度。

1.4 功能实现分析

1.4.1 图片表情识别

图片表情识别由 `expression_script.js` 实现。脚本定义了中性、高兴、难过、生气、害怕、厌恶、惊讶七类表情的中文标签，并按照检测框的横坐标对多人脸结果排序。每个人脸的表情概率由 `face-api.js` 返回，脚本遍历七类表情，选择概率最高的一类作为主表情，同时用进度条展示各类别分数。若图片中没有检测到人脸，页面会显示“未检测到人脸”的空状态提示。

该功能的特点是把视觉标注和结构化文本结果同时呈现：画布负责显示检测框和表情标签，侧边栏负责显示更细的概率分布。这样既能直观看到模型定位是否正确，也能观察不同表情类别之间的置信度差异。

1.4.2 年龄与性别估计

年龄与性别估计由 `estimation_script.js` 实现。页面上传图片后，脚本调用 `detectAllFaces(image)` 得到每张人脸的估计年龄、性别标签和性别置信度。年龄结果经过四舍五入后以“岁”为单位展示，性别标签被映射为“男性”或“女性”，并通过检测框标签叠加在图片上。

该模块体现了多任务人脸属性估计的基本流程：同一张输入图片先经过人脸检测和特征提取，再由属性估计网络输出年龄与性别信息。由于年龄和性别都属于外观属性估计，结果受光照、遮挡、姿态、图片清晰度和样本分布影响较大，因此在实际应用中更适合作为参考性信息，而不宜作为高风险自动决策的唯一依据。

1.4.3 摄像头实时追踪

摄像头追踪由 `webcam_script.js` 实现。脚本首先检查浏览器是否支持摄像头访问接口，并在页面中展示“等待授权”“正在连接”“摄像头已连接”等状态。当视频开始播放后，页面创建覆盖画布，并每隔 100 ms 执行一次检测流程：使用 `TinyFaceDetectorOptions` 检测视频帧中的人脸，继续提取关键点和表情，再清空画布并重绘检测框、关键点和表情结果。

实时模块还处理了两个交互细节。第一，脚本根据展示区域尺寸同步画布大小，窗口尺寸变化时重新匹配画布，避免检测结果与视频画面错位。第二，侧边栏结果设置了 280 ms 的刷新闻隔，并在结果未变化时跳过重复渲染，减少高频检测带来的 DOM 更新压力。这些处理使前端实时识别界面更平稳。

1.5 实验结果与效果

从功能完整性看，实验系统完成了三类典型人脸分析任务。图片表情识别页面能够在上传图片后绘制人脸框，并给出每张人脸的主表情及七类表情概率；年龄与性别估计页面能够显示检测框、估计年龄、性别标签和置信度；摄像头追踪页面能够在浏览器授权后持续处理视频帧，并实时叠加检测框、68 点关键点和表情结果。

从工程实现看，该系统结构清晰，HTML 页面主要负责布局和资源引入，具体推理逻辑由 `dist/` 下的脚本承担，模型权重和示例资源保存在本地目录。页面交互采用统一的“左侧控制与结果、右侧图像或视频舞台”结构，便于比较不同实验模块的输入输出差异。

从实验局限看，系统依赖浏览器端计算能力，实时识别帧率会受到设备性能和摄像头分辨率影响；模型采用预训练权重，不能保证对所有年龄段、光照条件、姿态和遮挡情况都有稳定表现；摄像头功能需要浏览器权限和安全上下文支持，部分环境下可能无法直接运行。此外，表情、年龄和性别结果都属于算法估计，报告中应将其理解为模型输出而非绝对真实标签。

1.6 本部分小结

本部分围绕 `face-recognition` 项目完成了浏览器端人脸分析实验说明。实验展示了 `face-api.js` 在静态网页中完成模型加载、图片推理、摄像头视频流处理和结果可视化的完整流程。通过对表情识别、年龄性别估计和实时追踪三个模块的分析，可以看出前端深度学习推理适合教学演示和轻量级交互原型，但在严肃应用中仍需要结合数据集评估、隐私保护、误差分析和系统性能测试进一步完善。

2 第二部分 后续内容

本报告第二部分用于补充其他实验或扩展分析内容。当前先保留章节结构，后续可根据课程要求继续撰写研究背景、方法设计、实验过程、结果分析和改进方向。

2.1 待补充内容

本节暂不展开。后续写作时，建议与第一部分保持一致的结构：先说明问题背景和目标，再介绍系统结构或算法流程，最后给出实验结果、局限性和改进方案。

参考文献

- [1] Vincent Muhler. face-api.js: JavaScript API for face detection and face recognition in the browser and nodejs with tensorflow.js. GitHub, 2018.
- [2] World Wide Web Consortium. Media Capture and Streams. W3C Recommendation.
- [3] Bootstrap Team. Bootstrap Documentation.